Machine Learning Fundamentals

SUPPORT VECTOR MACHINES

Andy Louwyck
andy.louwyck@vives.be
Academic year 2024 - 2025

hogeschool vives

What are support vector machines?

Support Vector Machine

- Abbreviated as SVM
- Very powerful and versatile machine learning model
- Can be used to solve linear and nonlinear problems:
  - classification
  - Regression

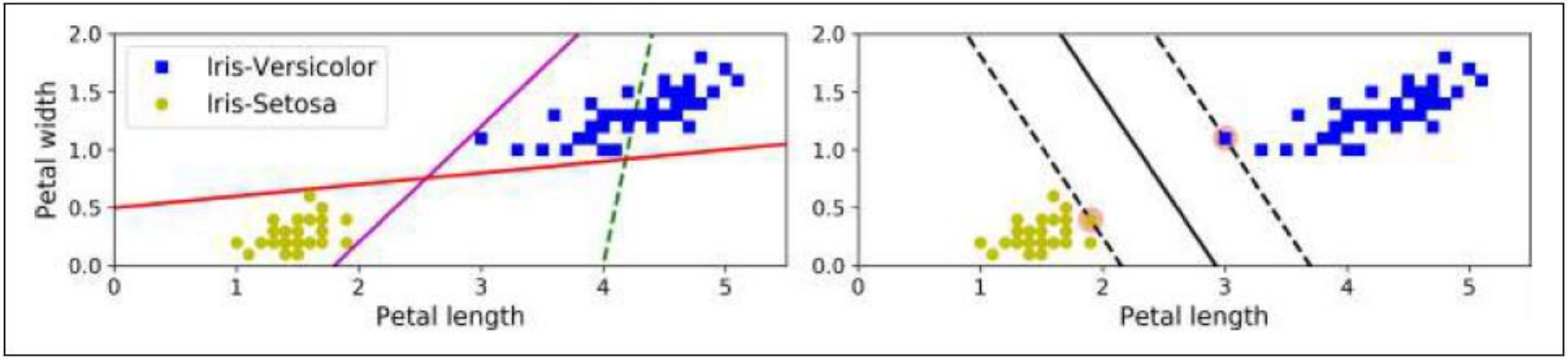
  = supervised learning
- But especially useful for classification tasks with small to medium sized datasets
- Can also be used for outlier detection

Linear SVM Classification

Linear Classification

- Already seen: perceptron and logistic regression
- Separating line as a decision boundary
- E.g. iris dataset:
  - Left – green dotted line: bad classifier
  - Left – solid lines: also perform well with new data?
  - Right: SVM tries to fit the widest possible street

  = large margin classification

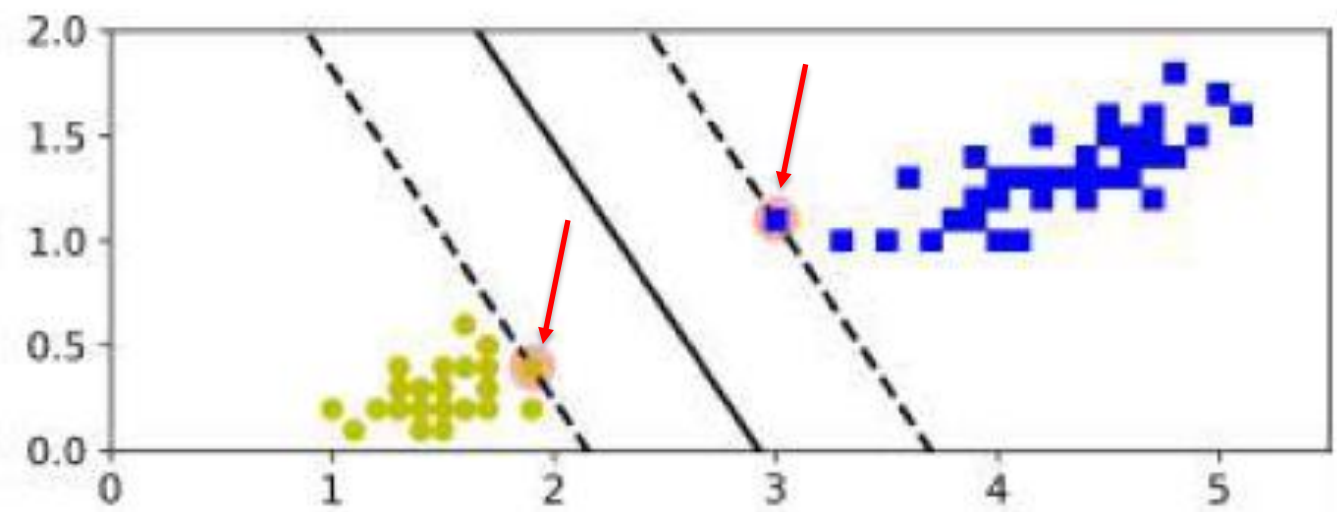


Support Vectors

- Data points outside the street have no influence

- The decision boundary is completely determined by the data points on the edge of the street

  = the support vectors



Feature Scaling!

SVMs are sensitive to the feature scales, as you can see in Figure 5-2: on the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal. After feature scaling (e.g., using Scikit-Learn's `StandardScaler`), the decision boundary looks much better (on the right plot).

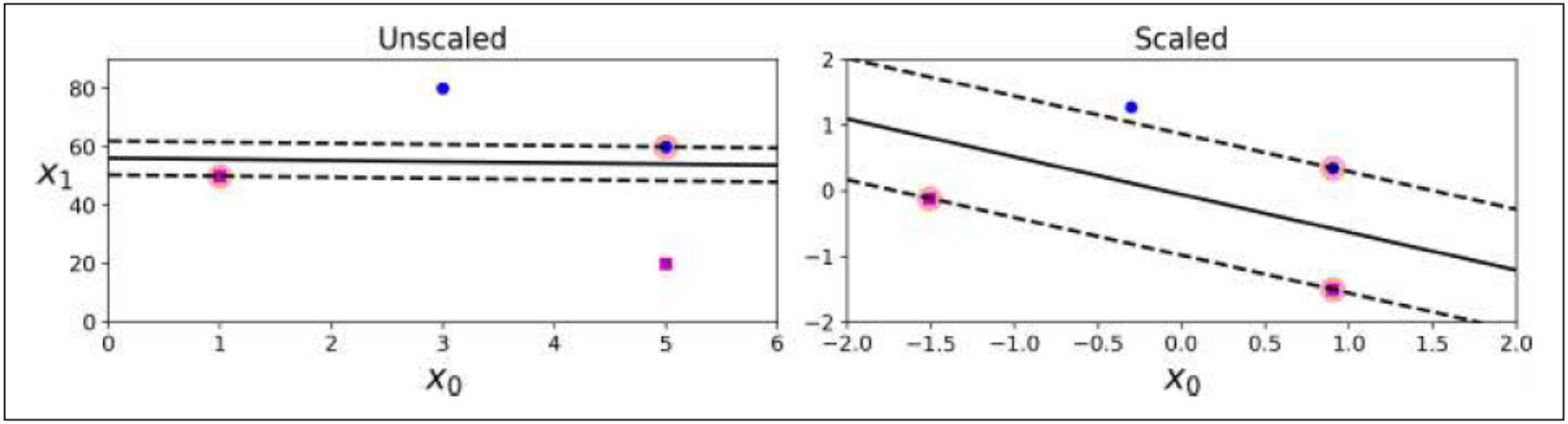
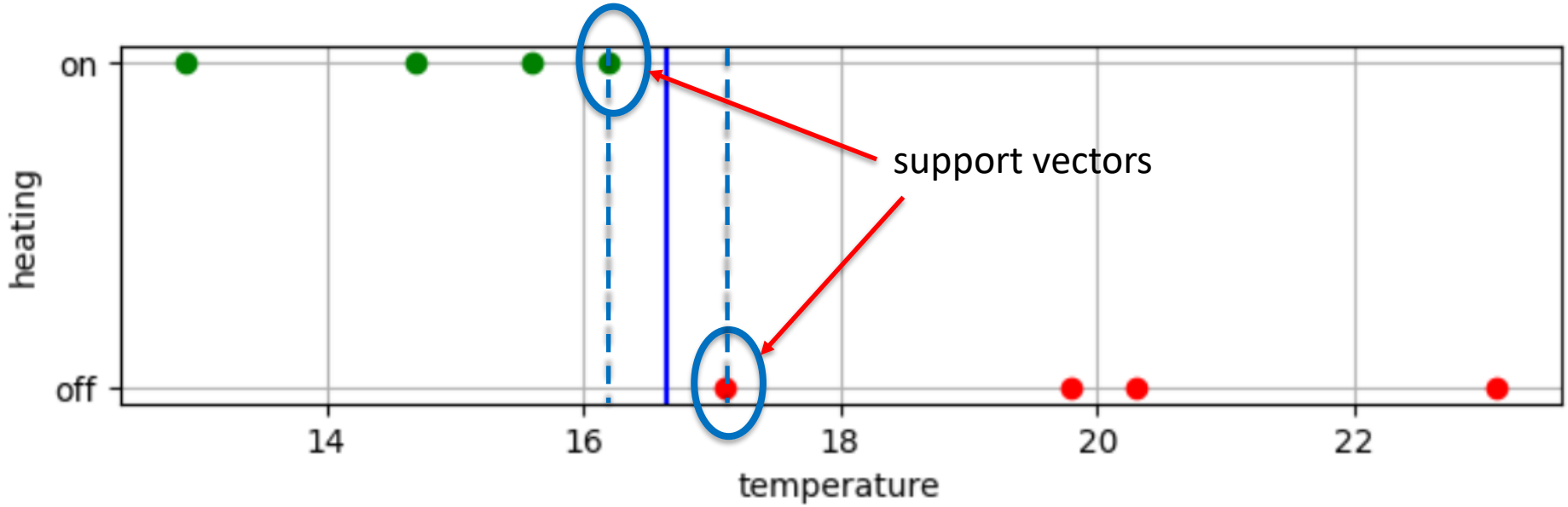


Figure 5-2. Sensitivity to feature scales

Example: thermostat

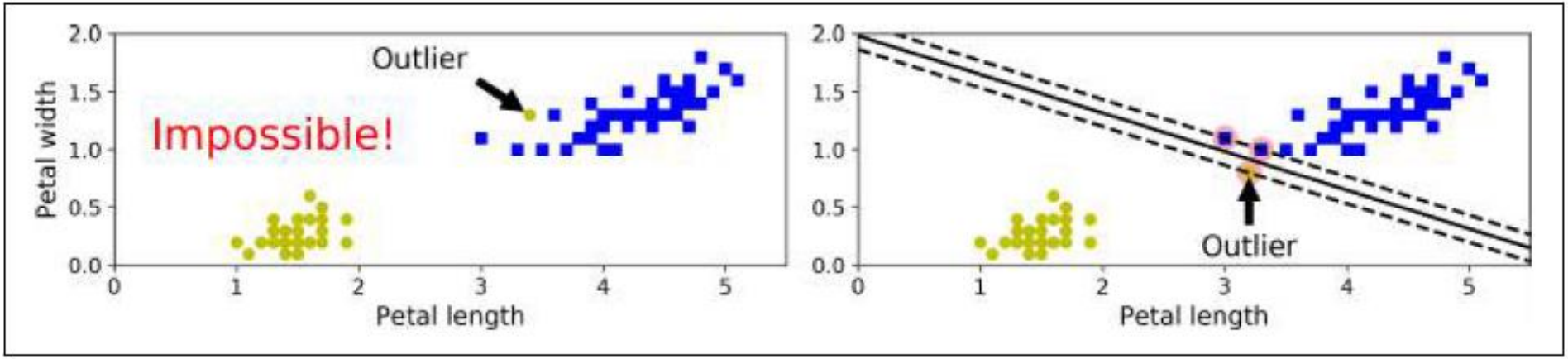
```
max_temperature_on = table[table.heating=='on']['temperature'].max()
min_temperature_off = table[table.heating=='off']['temperature'].min()
threshold = (max_temperature_on + min_temperature_off) / 2
print(f'maximum temperature if heating is on: {max_temperature_on}°C')
print(f'minimum temperature if heating is off: {min_temperature_off}°C')
print(f'threshold is {threshold}°C')
```

```
maximum temperature if heating is on: 16.2°C
minimum temperature if heating is off: 17.1°C
threshold is 16.65°C
```



Hard Margin Classification

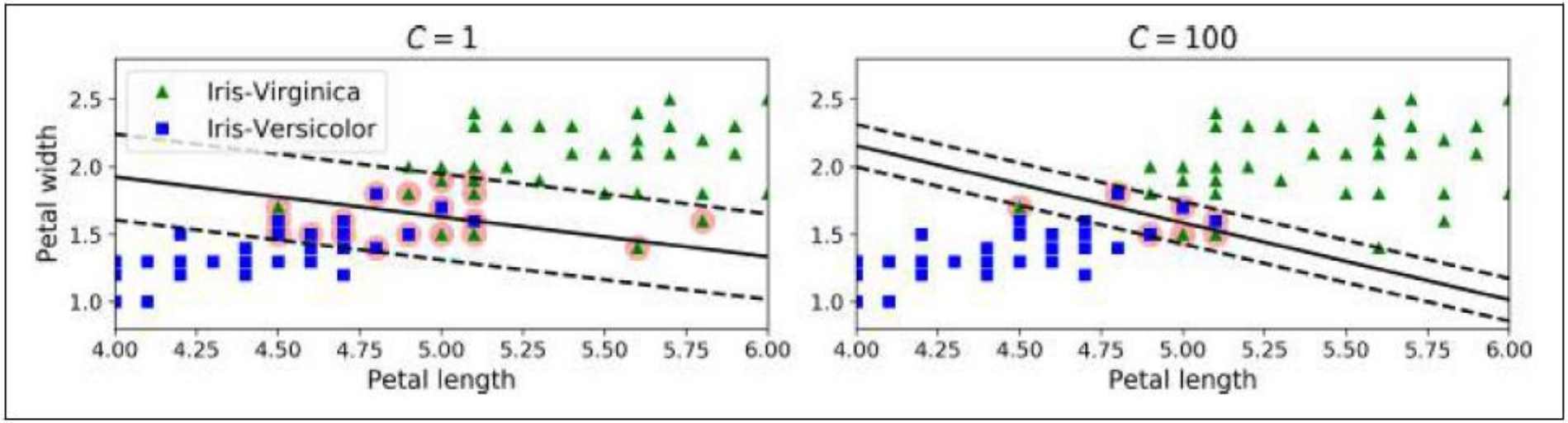
- All data points must be outside the street
- Only works if the dataset is linearly separable
- Is very sensitive to outliers



- Solution: soft margin classification!

Soft Margin Classification

- Finding a balance between
  - keeping the street as wide as possible
  - having as few data points as possible in the street or on the wrong side of the street
- Hyperparameter C: the smaller, the wider the street
- Overfitting? Regularize SVM by shrinking C!



Scikit-Learn

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # petal length, petal width
y = (iris["target"] == 2).astype(np.float64) # Iris-Virginica
C = 1  # hyperparameter C
svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=C, loss="hinge")),
])
svm_clf.fit(X, y)
svm_clf.predict([[5.5, 1.7]]).item()
```

```
1.0
```

Unlike Logistic Regression classifiers, SVM classifiers do not output probabilities for each class.

Attention!

The `LinearSVC` class regularizes the bias term, so you should center the training set first by subtracting its mean. This is automatic if you scale the data using the `StandardScaler`. Moreover, make sure you set the `loss` hyperparameter to `"hinge"`, as it is not the default value. Finally, for better performance you should set the `dual` hyperparameter to `False`, unless there are more features than training instances (we will discuss duality later in the chapter).

hogeschool vives

Alternatives

```
from sklearn.svm import SVC

svc_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", SVC(C=C, kernel="linear")),
])
svc_clf.fit(X, y)
svc_clf.predict([[5.5, 1.7]]).item()
```

slower than LinearSVC

```
1.0
```

```
from sklearn.linear_model import SGDClassifier

m = len(y)  # number of samples
sgd_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", SGDClassifier(alpha=1/(m*C), loss="hinge")),
])
svc_clf.fit(X, y)
svc_clf.predict([[5.5, 1.7]]).item()
```

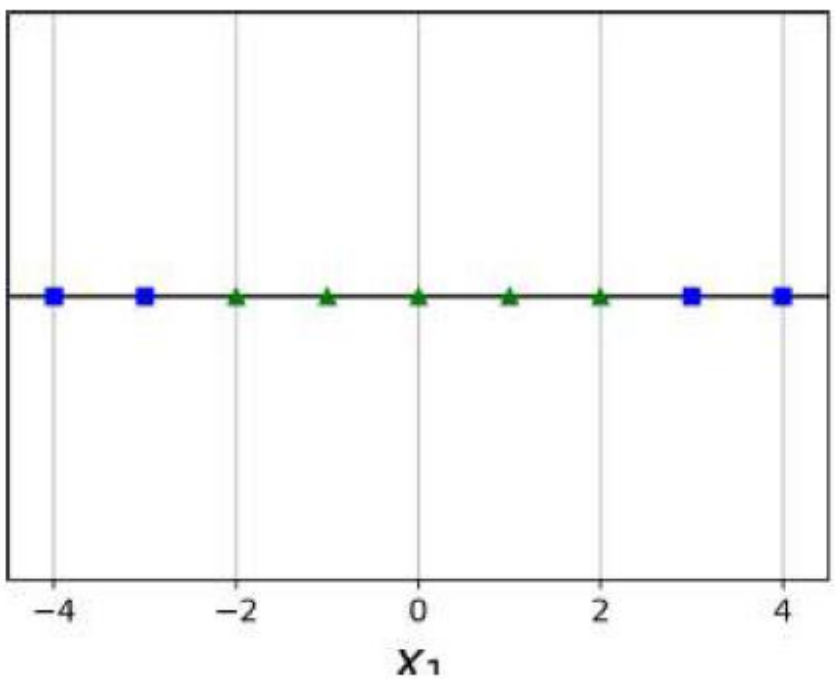
- slower convergence than LinearSVC
- more interesting for large data sets

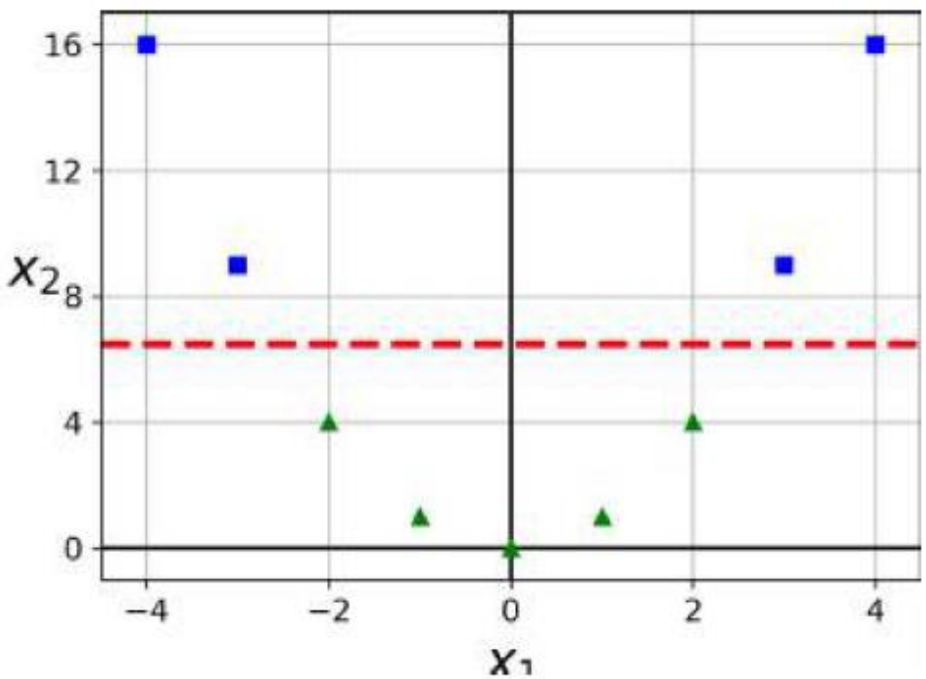
```
1.0
```

Nonlinear SVM Classification

Polynomial Features

- Add X^2, X^3, …, X^n as features
- Just like in polynomial regression
- For example, $X_2 = (X_1)^2$:



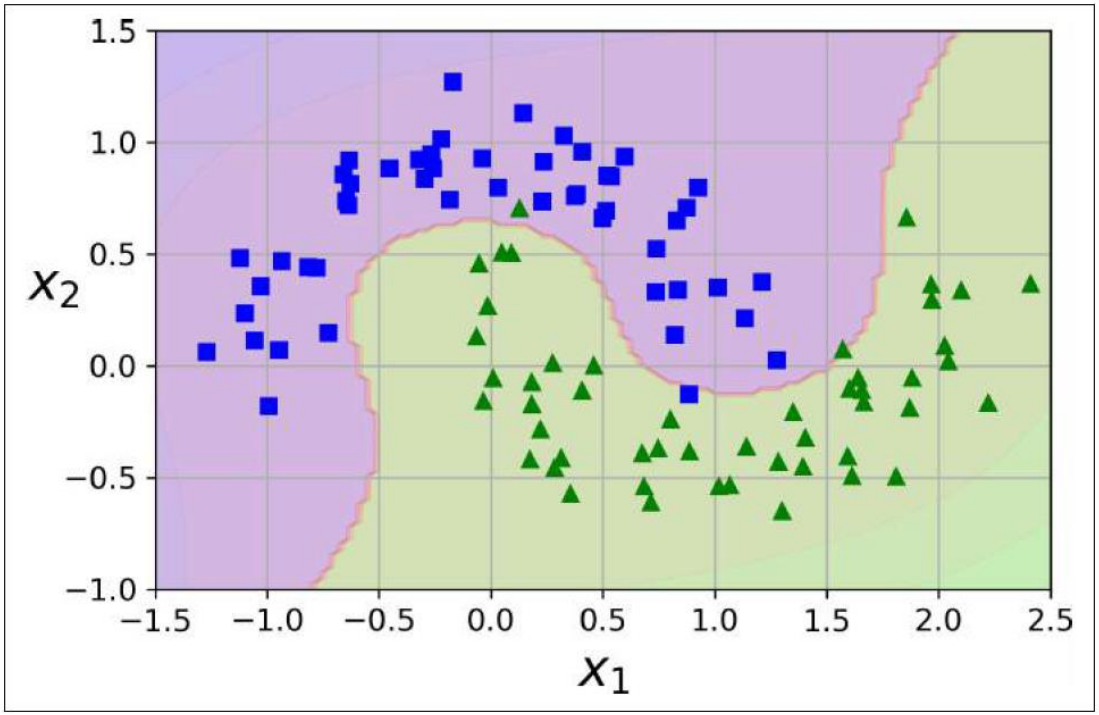


Example

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
        ("svm_clf", LinearSVC(C=10, loss="hinge"))
    ])

polynomial_svm_clf.fit(X, y)
```



Polynomial Kernel

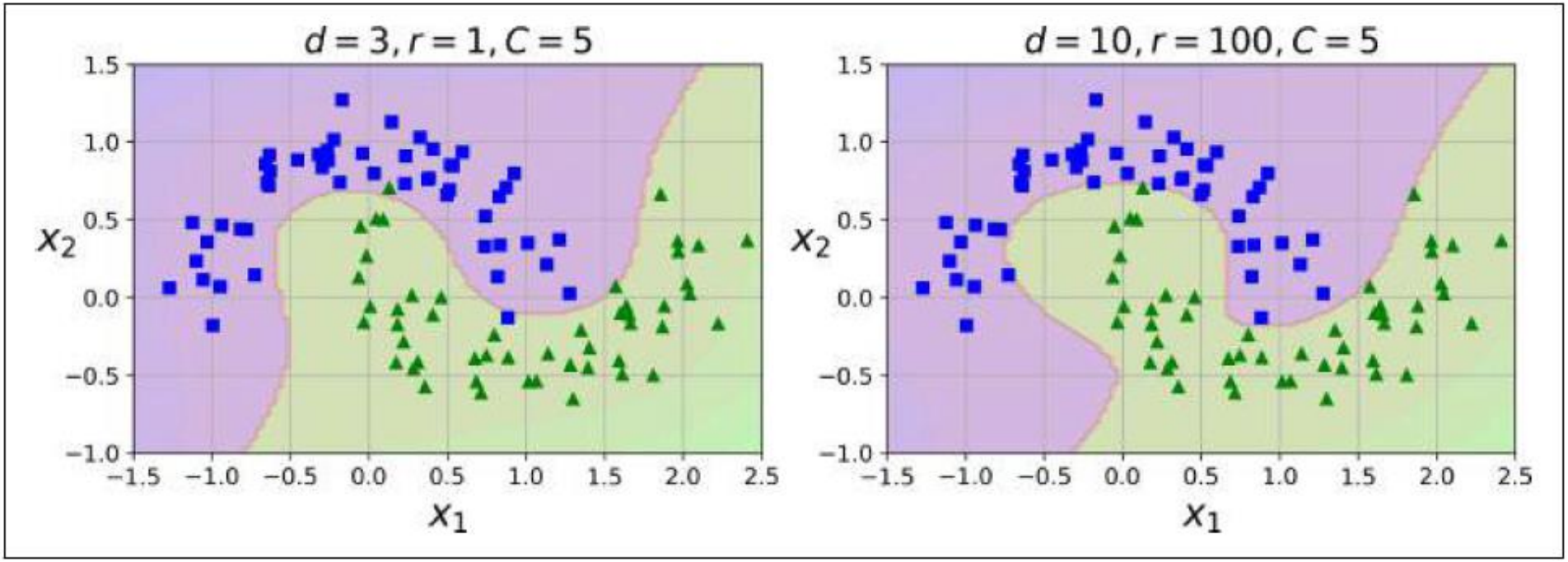
- Adding polynomial features can slow down training
- With SVM you can apply the mathematical kernel trick!

  = same result without adding extra features

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```

- Hyperparameters:
  - Degree d: highest power in the polynomial
  - coef0 = r: determines balance between influence of low power terms versus high power terms

Overfitting

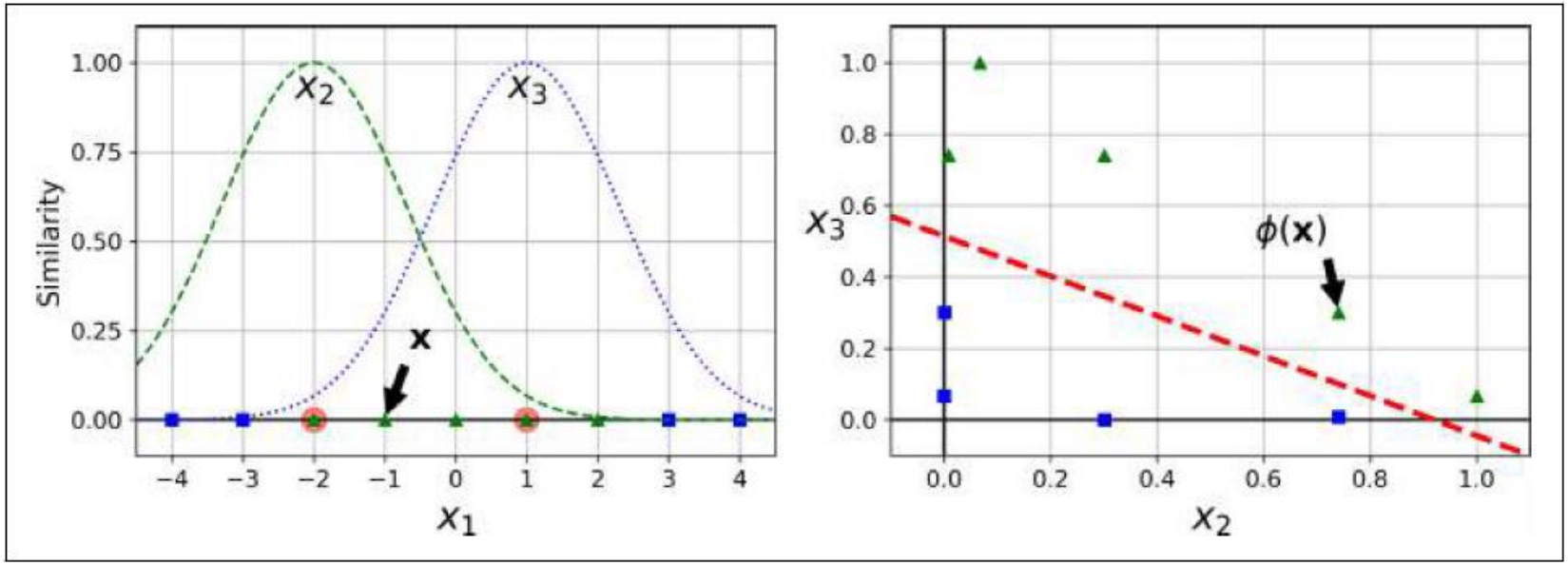


A common approach to find the right hyperparameter values is to use grid search (see Chapter 2). It is often faster to first do a very coarse grid search, then a finer grid search around the best values found. Having a good sense of what each hyperparameter actually does can also help you search in the right part of the hyperparameter space.

Similarity Features

- Adding features determined by similarity function
- Gaussian Radial Basis Function (RBF): $\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma \| \mathbf{x} - \ell \|^2\right)$

  = transformed distance between features **x** and ℓ
- Landmarks ℓ are chosen (e.g. the data points themselves)
- For example, landmarks -2 and 1:



Gaussian RBF Kernel

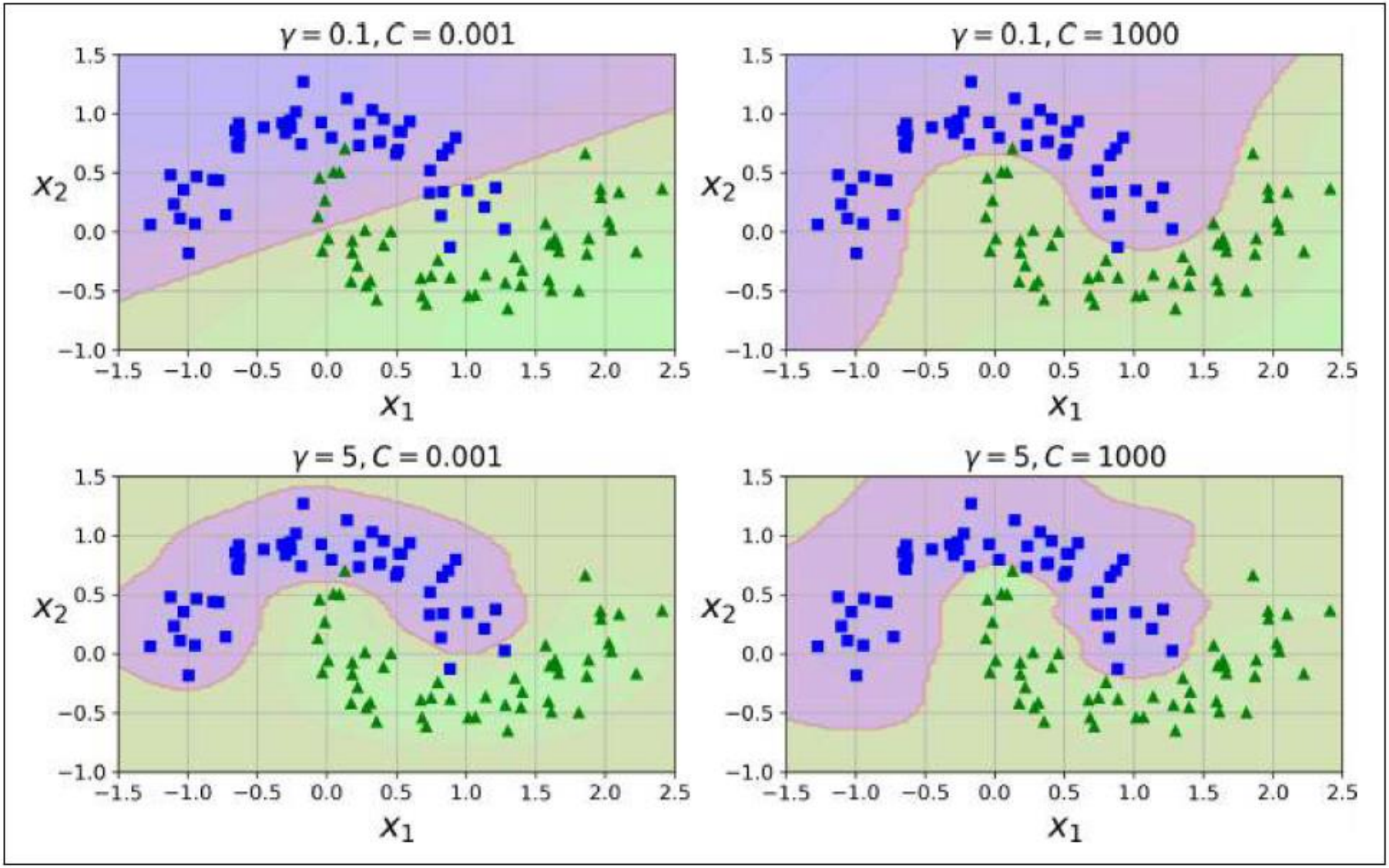
The kernel trick can also be applied here!

```
rbf_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
    ])
rbf_kernel_svm_clf.fit(X, y)
```

gamma = γ: is also a hyperparameter

- The larger, the narrower the bell-shaped curve, and the more irregular the decision boundary
- Regularization parameter: reduce when overfitting!

Influence of Hyperparameters



Other Kernels?

There are other kernels; however, they are rare…

(e.g. string kernel)

With so many kernels to choose from, how can you decide which one to use? As a rule of thumb, you should always try the linear kernel first (remember that `LinearSVC` is much faster than `SVC(kernel="linear")`), especially if the training set is very large or if it has plenty of features. If the training set is not too large, you should try the Gaussian RBF kernel as well; it works well in most cases. Then if you have spare time and computing power, you can also experiment with a few other kernels using cross-validation and grid search, especially if there are kernels specialized for your training set's data structure.

hogeschool vives

Computational Complexity

- **LinearSVC:**
  - liblinear solver (cfr LogisticRegression)
  - optimized for linear SVM
- **SVC:**
  - libsvm solver
  - kernel trick
- **SGDClassifier:**
  - Stochastic Gradient Descent as an optimizer
  - out-of-core training possible with very large datasets

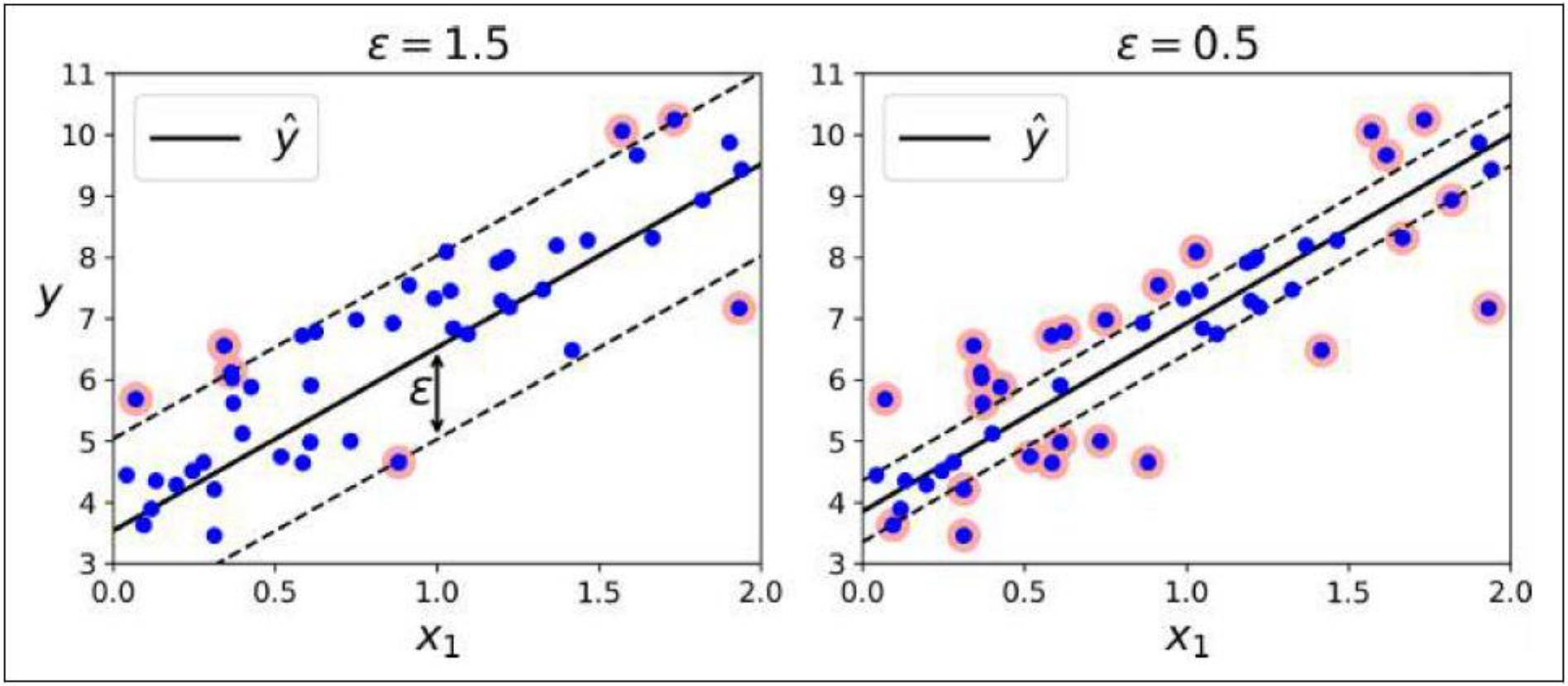
Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
`LinearSVC`	$O(m \times n)$	No	Yes	No
`SGDClassifier`	$O(m \times n)$	Yes	Yes	No
`SVC`	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

SVM Regression

What and how?

- SVM can also solve linear and nonlinear regression tasks
- The goal is different in regression: trying to get as many data points as possible within the street edges
- The width of the street is controlled by hyperparameter ϵ
- Model is ϵ-insensitive: adding data points within the street edges does not affect the predictions because the loss is zero for these points

Influence of ϵ



- Points inside the street: loss = 0
- Points outside the street: loss > 0

Scikit-Learn

- Linear problems:

```
from sklearn.svm import LinearSVR

svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```

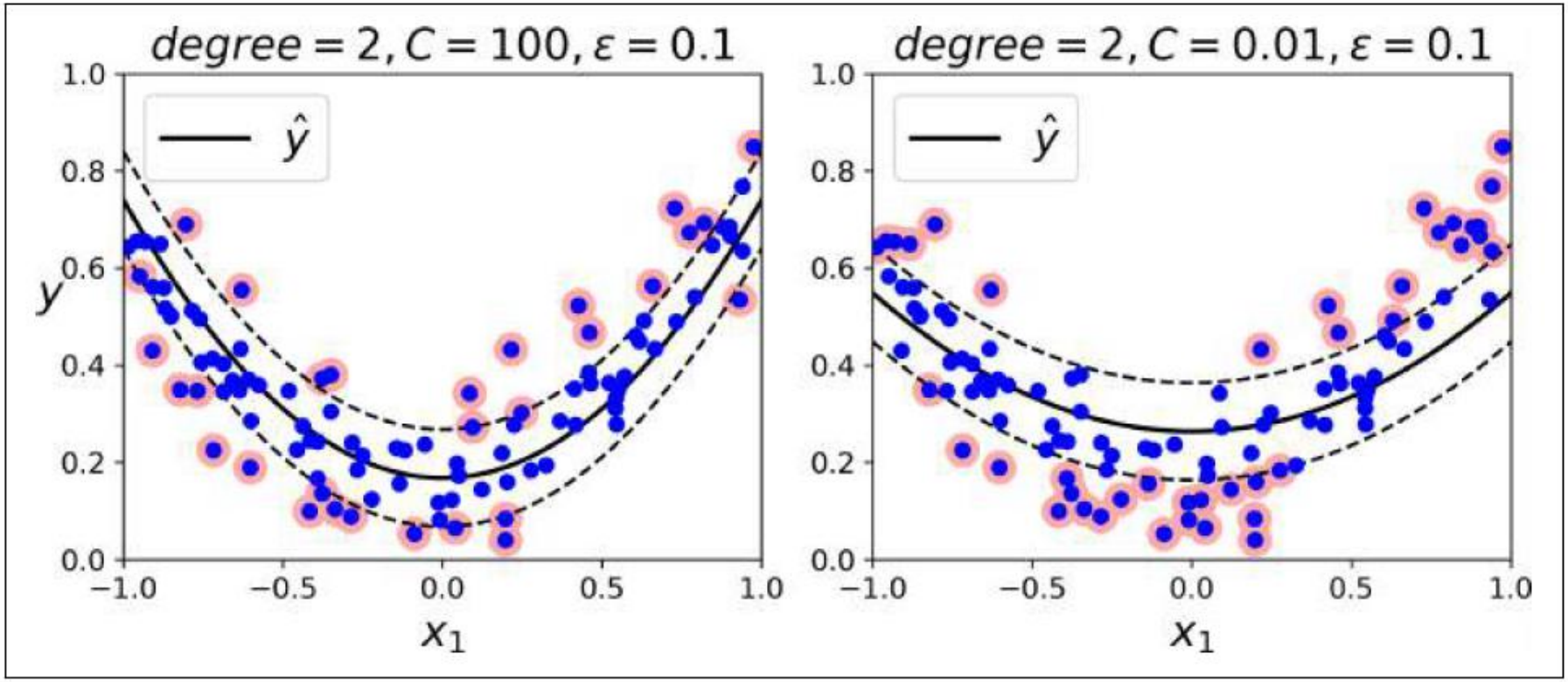
- Nonlinear problems: applying kernel

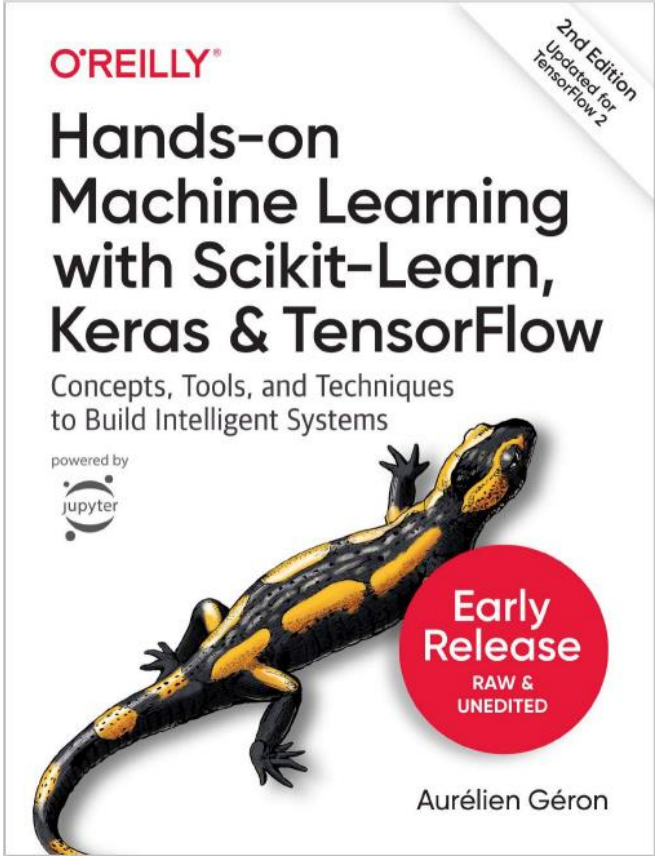
```
from sklearn.svm import SVR

svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg.fit(X, y)
```

- Like classification: LinearSVC and SVC

Example: Quadratic problem



Reference

Aurélien Géron (2019). Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow – Concepts, Tools, and Techniques to Build Intelligent Systems (2nd edition): Chapter 5 – Support Vector Machines.